

INF1022 - Analisadores Léxicos e Sintáticos

Linguagens Livres de Contexto

Edward Hermann Haeusler Vitor Pinheiro

PUC-Rio

Departamento de Informática

Equivalência de LLC e NPDAs (I)

Teorema

Seja $\mathcal{G} = \langle \Sigma, \Gamma, S, \mathcal{R} \rangle$ uma Gramática Livre de Contexto. Existe um NPDA $\mathcal{A}$, tal que $\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{G})$.

Equivalência de LLC e NPDAs (I)

Teorema

Seja $\mathcal{G} = \langle \Sigma, \Gamma, S, \mathcal{R} \rangle$ uma Gramática Livre de Contexto. Existe um NPDA $\mathcal{A}$, tal que $\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{G})$.

Prova Construa, $\mathcal{A} = \langle \{q_0\}, \Sigma, \Gamma \cup \Sigma, \delta, q_0, S, \{q_0\} \rangle$ e δ é definido como:

$$\begin{array}{ll} (1) (q_0, \gamma) \in \delta(q_0, \epsilon, N) & \text{se e somente se } N \rightarrow \gamma \in \mathcal{R} \text{ e } N \in \Gamma \\ (2) \delta(q_0, \sigma, \sigma) = (q_0, \epsilon) & \text{se e somente se } \sigma \in \Sigma \end{array}$$

- (1) Regra de extensão
- (2) Regra de verificação

Equivalência de LLC e NPDAs (I)

Prova Construa, $\mathcal{A} = \langle \{q_0\}, \Sigma, \Gamma \cup \Sigma, \delta, q_0, S, \{q_0\} \rangle$ e δ é definido como:

$$\begin{array}{ll} (1) (q_0, \gamma) \in \delta(q_0, \epsilon, N) & \text{se e somente se } N \rightarrow \gamma \in \mathcal{R} \text{ e } N \in \Gamma \\ (2) \delta(q_0, \sigma, \sigma) = (q_0, \epsilon) & \text{se e somente se } \sigma \in \Sigma \end{array}$$

- (1) Regra de extensão
- (2) Regra de verificação

$$\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{G})$$

. Seja $\omega \in \mathcal{L}(G)$ e seja $S \Rightarrow \omega_1 \Rightarrow \omega_2 \dots \Rightarrow \omega_k = \omega$ uma derivação mais a esquerda (leftmost) de ω , então^a:

- Seja $(S, \omega) \vdash_{EX} (\omega_1, \omega) \dots \vdash_{EX} (\omega_k, \omega)$. Obs: lembre-se que $\omega_k = \omega$.
- Substitua, da esquerda para a direita, cada par de configurações $(\sigma_1 \dots \sigma_j N \rho, \sigma_1 \dots \sigma_j \omega')$ por $(\sigma_1 \dots \sigma_j N \rho, \sigma_1 \dots \sigma_j \omega')$ $\vdash_V$ $(\sigma_2 \dots \sigma_j N \rho, \sigma_2 \dots \sigma_j \omega') \vdash_V (\sigma_j N \rho, \sigma_j \omega') \vdash_{EX} (\gamma \rho, \omega')$ e apague $\sigma_1 \dots \sigma_j$ do prefixo das configurações seguintes a este par substituído;
- A última configuração depois do efeito dos passos acima será da forma $(\sigma_1 \dots \sigma_k, \sigma_1 \dots \sigma_k)$, com $\sigma_i \in \Sigma_G$. Substitua-a pela sequência $(\sigma_1 \dots \sigma_k, \sigma_1 \dots \sigma_k) \vdash_V (\sigma_2 \dots \sigma_k, \sigma_2 \dots \sigma_k) \dots \vdash_V (\sigma_k, \sigma_k) \vdash_V (\epsilon, \epsilon)$

● A última configuração sendo (ϵ, ϵ) indica uma computação no NPDA definida aceitando

Equivalência de LLC e NPDAs (I)

Exemplo: Gramática $\mathcal{G}$ para palíndromos pares

$\mathcal{R}$	(1)	(2)	(3)
$S \rightarrow$	aSa	bSb	ϵ

Derivação mais a esquerda de $abba$ por $\mathcal{G}$ acima:

$$S \Rightarrow aSa \Rightarrow abSba \Rightarrow abba$$

[1] Sequência de configurações de aceitação de $abba$ no NPDA $\mathcal{A}_{\mathcal{G}}$, primeiro ítem:

$$(S, abba) \vdash_{EX} (aSa, abba) \vdash_{EX} (abSba, abba) \vdash_{EX} (abba, abba)$$

[2] Aplicações subsequentes de obtenção da sequência de computação, segundo ítem:

$$(S, abba) \vdash_{EX} (aSa, abba) \vdash_V (Sa, bba) \vdash_{EX} (bSba, bba) \vdash_{EX} (bba, bba)$$

$$(S, abba) \vdash_{EX} (aSa, abba) \vdash_V (Sa, bba) \vdash_{EX} (bSba, bba) \vdash_V$$

$$(Sba, ba) \vdash_{EX} (ba, ba) \vdash_V (a, a) \vdash_V (\epsilon, \epsilon)$$

Exemplo II mais concreto

Gramática :

$$\begin{array}{lcl} E & \rightarrow & E + T \quad | \quad T \\ T & \rightarrow & T * F \quad | \quad F \\ F & \rightarrow & (E) \quad | \quad a \end{array}$$

gera $a + a * a$ e $\mathcal{A}_G$ o aceita com base na sua derivação mais a esquerda.

pilha	(resto da) entrada	derivação esquerda
E	a+a*a	E
E+T	a+a*a	$\Rightarrow$ E+T
T+T	a+a*a	$\Rightarrow$ T+T
F+T	a+a*a	$\Rightarrow$ F+T
a+T	a+a*a	$\Rightarrow$ a+T
+T	+a*a	
T	a*a	
T*F	a*a	$\Rightarrow$ a+T*F
F*F	a*a	$\Rightarrow$ a+F*F
a*F	a*a	$\Rightarrow$ a+a*F
*F	*a	
F	a	
a	a	$\Rightarrow$ a+a*a
ϵ	ϵ	

Formas especiais de Gramáticas LC

Forma Normal de Chomsky: Regras na forma:

$N \rightarrow N_1 N_2$ ou na forma
 $N \rightarrow a$ ou somente para S
 $S \rightarrow \epsilon$

Formas especiais de Gramáticas LC

Forma Normal de Chomsky: Regras na forma:

$$\begin{array}{ll} N \rightarrow N_1 N_2 & \text{ou na forma} \\ N \rightarrow a & \text{ou somente para } S \\ S \rightarrow \epsilon \end{array}$$

Forma Normal de Greibach: Regras na forma:

$$\begin{array}{ll} N \rightarrow a N_1 N_2 \dots N_k & \text{ou na forma} \\ N \rightarrow a & \text{ou somente para } S \\ S \rightarrow \epsilon \end{array}$$

Manipulações de gramática

Alterações efetuadas em gramáticas que preservam a linguagem, mas, em geral tornam a gramática mais adequada à análise sintática.

Problema de recursividade:

- Para que se possa construir um analisador sintático descendente, uma gramática não pode ter regras recursivas à esquerda (diretas ou indiretas).
- Uma gramática é recursiva à esquerda se tem produções da forma: $N \rightarrow N\delta$ ou $N \rightarrow^+ N\delta$.

Nesse caso, qualquer algoritmo que implemente um analisador descendente vai entrar em ciclo ("loop") infinito.

Manipulações de gramática

Alterações efetuadas em gramáticas que preservam a linguagem, mas, em geral tornam a gramática mais adequada à análise sintática.

1 Transformação de recursão a esquerda:

$$N \rightarrow N\gamma_1 \mid \dots \mid N\gamma_k$$

$$N \rightarrow \delta_1 \mid \dots \mid \delta_r$$

em recursão a direita:

$$N \rightarrow \delta_1 \mid \dots \mid \delta_r$$

$$N \rightarrow \delta_1 N' \mid \dots \mid \delta_r N'$$

$$N' \rightarrow \gamma_1 N' \mid \dots \mid \gamma_k N' \mid \gamma_1 \mid \dots \mid \gamma_k$$

2 Fatoração: $N \rightarrow \gamma A_1 \mid \dots \mid \gamma A_n$

fatora como:

$$\begin{aligned} N &\rightarrow \gamma B \\ B &\rightarrow A_1 \mid \dots \mid A_n \end{aligned}$$

3 Criação de não-terminais e estruturação de regras: $N \rightarrow \gamma_1 \gamma_2$ se

$$N \rightarrow N_1 N_2$$

transforma em $N_1 \rightarrow \gamma_1$

$$N_2 \rightarrow \gamma_2$$

Manipulações de gramática

Alterações efetuadas em gramáticas que preservam a linguagem, mas, em geral tornam a gramática mais adequada à análise sintática.

- 1 Transformação de recursão a esquerda:

$$N \rightarrow N\gamma_1 \mid \dots \mid N\gamma_k$$

$$N \rightarrow \delta_1 \mid \dots \mid \delta_r$$

em recursão a direita:

$$N \rightarrow \delta_1 \mid \dots \mid \delta_r$$

$$N \rightarrow \delta_1 N' \mid \dots \mid \delta_r N'$$

$$N' \rightarrow \gamma_1 N' \mid \dots \mid \gamma_k N' \mid \gamma_1 \mid \dots \mid \gamma_k$$

- 2 Fatoração: $N \rightarrow \gamma A_1 \mid \dots \mid \gamma A_n$

fatora como:

$$\begin{array}{l} N \rightarrow \gamma B \\ B \rightarrow A_1 \mid \dots \mid A_n \end{array}$$

- 3 Criação de não-terminais e estruturação de regras: $N \rightarrow \gamma_1 \gamma_2$ se

$$N \rightarrow N_1 N_2$$

transforma em $N_1 \rightarrow \gamma_1$

$$N_2 \rightarrow \gamma_2$$

Manipulações de gramática

Alterações efetuadas em gramáticas que preservam a linguagem, mas, em geral tornam a gramática mais adequada à análise sintática.

- 1 Transformação de recursão a esquerda:

$$N \rightarrow N\gamma_1 \mid \dots \mid N\gamma_k$$

$$N \rightarrow \delta_1 \mid \dots \mid \delta_r$$

em recursão a direita:

$$N \rightarrow \delta_1 \mid \dots \mid \delta_r$$

$$N \rightarrow \delta_1 N' \mid \dots \mid \delta_r N'$$

$$N' \rightarrow \gamma_1 N' \mid \dots \mid \gamma_k N' \mid \gamma_1 \mid \dots \mid \gamma_k$$

- 2 Fatoração: $N \rightarrow \gamma A_1 \mid \dots \mid \gamma A_n$

fatora como:

$$\begin{array}{l} N \rightarrow \gamma B \\ B \rightarrow A_1 \mid \dots \mid A_n \end{array}$$

- 3 Criação de não-terminais e estruturação de regras: $N \rightarrow \gamma_1 \gamma_2$ se

$$N \rightarrow N_1 N_2$$

transforma em $N_1 \rightarrow \gamma_1$

$$N_2 \rightarrow \gamma_2$$

Manipulações de gramática

Exemplo de eliminação de recursividade a esquerda. Ou seja, transformar uma recursividade à esquerda em recursividade à direita.

① Gramática recursiva à esquerda:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow ID \mid (E)$$

Transformando em recursão a direita:

$$E \rightarrow T + E' \quad E' \rightarrow +T E' \mid \epsilon$$

$$T \rightarrow F T' \quad T' \rightarrow *F T' \mid \epsilon$$

$$F \rightarrow ID \mid (E)$$

Por manipulação gramatical, tem-se

Teorema

Toda LLC possui uma gramática em forma normal de Greibach.

Prova Retira-se as regras da forma $A \rightarrow \epsilon$, exceto $S \rightarrow \epsilon$, retira-se recursão à esquerda ($A \rightarrow A\gamma$) de todas as regras e troca-se o não-terminal mais a esquerda em uma regra pela sua definição. É necessário uma certa disciplina.

Exemplo com uma gramática concreta

$$\begin{array}{lcl} \text{Gramática : } & E & \rightarrow E + T \mid T \\ & T & \rightarrow T * F \mid F \\ & F & \rightarrow (E) \mid a \end{array}$$

Substituição a definição de F em parte da regra do T

$$\begin{array}{lcl} \text{Gramática : } & E & \rightarrow E + T \mid T \\ & T & \rightarrow T * F \mid (E) \mid a \\ & F & \rightarrow (E) \mid a \end{array}$$

Exemplo com uma gramática concreta

$$\begin{array}{lcl} \text{Gramática : } E & \rightarrow & E + T \quad | \quad T \\ T & \rightarrow & T * F \quad | \quad (E) \quad | \quad a \\ F & \rightarrow & (E) \quad | \quad a \end{array}$$

Removendo a recursão à esquerda das regras do T

$$\begin{array}{lcl} \text{Gramática : } E & \rightarrow & E + T \quad | \quad T \\ T & \rightarrow & (E) \quad | \quad a \\ T & \rightarrow & (E)T' \quad | \quad aT' \\ T' & \rightarrow & *FT' \quad | \quad *F \\ F & \rightarrow & (E) \quad | \quad a \end{array}$$

Exemplo com uma gramática concreta

Gramática :	E	$\rightarrow$	$E + T$		T
	T	$\rightarrow$	(E)		a
	T	$\rightarrow$	$(E)T'$		aT'
	T'	$\rightarrow$	$*FT'$		$*F$
	F	$\rightarrow$	(E)		a

Substituindo T e removendo a recursão à esquerda das regras do E

Gramática :	E	$\rightarrow$	(E)		a		$(E)T'$		aT'
	E	$\rightarrow$	$(E)E'$		aE'		$(E)T'E'$		$aT'E'$
	E'	$\rightarrow$	$+TE'$		$+T$				
	T	$\rightarrow$	(E)		a				
	T	$\rightarrow$	$(E)T'$		aT'				
	T'	$\rightarrow$	$*FT'$		$*F$				
	F	$\rightarrow$	(E)		a				

Teorema

Toda LLC possui uma gramática na forma normal de Chomsky

Prova

A partir de uma gramática na forma normal de Greibach, fatora-se binariamente cada regra da gramática, obtendo-se uma formal normal de Chomsky.

Quantas regras, em função do número de não-terminais, possui, no máximo, uma gramática em forma normal de Chomsky ??

Um guia para ajudar a converter para a forma normal de Chomsky ou Greibach

1 Eliminação de regras $A \rightarrow \epsilon$

- ▶ Remover a regra.
- ▶ Para toda regra $R \rightarrow uAv$ adicionar $R \rightarrow uv$
- ▶ NOTA: Fazer isso para cada ocorrência individual de A. Ex: se $R \rightarrow uAvAw$, deve-se acrescentar 3 regras: $R \rightarrow uvAw$, $R \rightarrow uAvw$, $R \rightarrow uvw$
- ▶ Repetir até eliminar todas as ocorrências.

2 Remoção das regras unitárias $A \rightarrow B$

- ▶ Remover a regra.
- ▶ Para toda regra $B \rightarrow \delta$, acrescentamos $A \rightarrow \delta$, a menos que essa seja uma regra unitária já removida.
- ▶ Repetir para todas as regras unitárias.

3 Nesta etapa podemos tanto converter as regras remanescentes para FNC (Chomsky) ou para FNG (Greibach).

Análise preditiva recursiva descendente

Também Análise recursiva descendente ou simplesmente preditiva descendente. Só é possível quando a gramática em forma normal de Greibach é determinística. Sempre produz uma derivação mais a esquerda, Cada não terminal gera um código de acordo com as suas produções: Se $A \rightarrow aA_1A_2|bB_1B_2B_3|C_1C_2$ são todas as regras para A , então:

```
% char c é a variável que armazena símbolo recém lido
void A()
    If (c="a") {read(c);A1();A2();}
        elseif (c="b") {read(c);B1();B2();B3();}
            else {C1();C2();};

void B1()
    If ...
void B2() ..
```

Não pode haver recursão a esquerda nem ϵ -regras. Invocação do parser: "read(c);S();" , onde S é o procedimento associado ao símbolo inicial da gramática

De APNDs para gramáticas livres de contexto (GLCs)

Teorema Para todo *APND* $\mathcal{A}$ existe um *APND* $\mathcal{U}_{\mathcal{A}}$ com somente um estado e $\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{U}_{\mathcal{A}})$

Dado um *APND* $\mathcal{U}$ com somente um estado só aplica-se o mapeamento inverso de GLCs para *APNDs* para obter uma gramática livre de contexto que gera $\mathcal{L}(\mathcal{U})$, isto é :

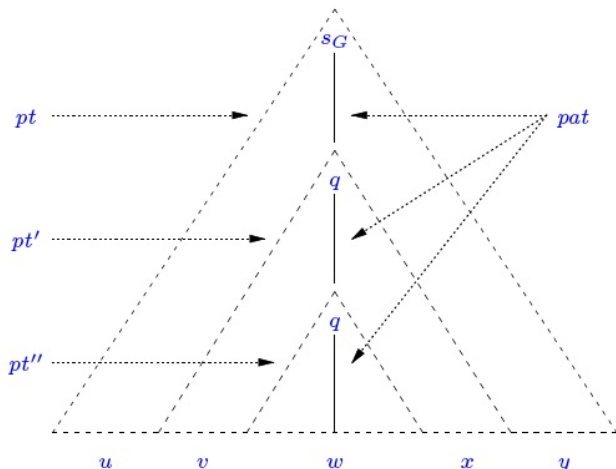
Teorema Toda linguagem aceita por *APND* é livre de contexto

Lema Seja L LC. Existe $k > 0$, tal que, para $\gamma \in L$, $|\gamma| \geq k$, existem $u, v, z, x, y \in \Sigma$ tal que $uvzxy = w$, $|vzx| \leq k$, $vx \neq \epsilon$ e para $i \in \text{Nat}$, $uv^i zx^i y \in L$.

Ou seja, dado a escolha de uma palavra w , qualquer que seja a divisão feita usando $w = uv^i zx^i y$ a cadeia gerada para qualquer i não deve pertencer a linguagem.

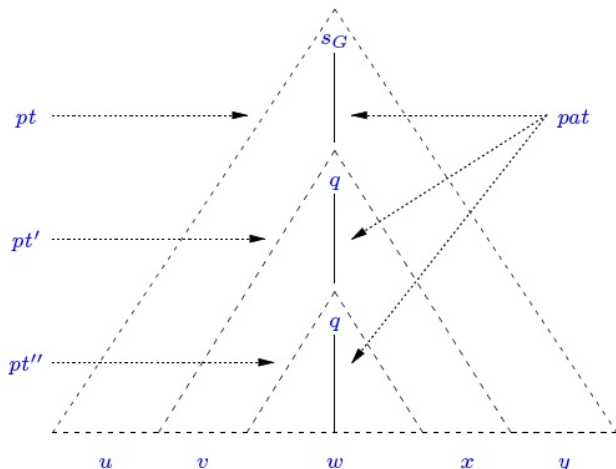
Lema do Bombeamento para LLC

Lema Seja L LC. Existe $k > 0$, tal que, para $\gamma \in L$, $|\gamma| \geq k$, existem $u, v, z, x, y \in \Sigma$ tal que $uvzxy = w$, $|vzx| \leq k$, $vx \neq \epsilon$ e para $i \in \mathbb{N}$, $uv^i z x^i y \in L$. **Prova** - G uma gramática LC para L , r o número de regras em G , $k > r + 1$.



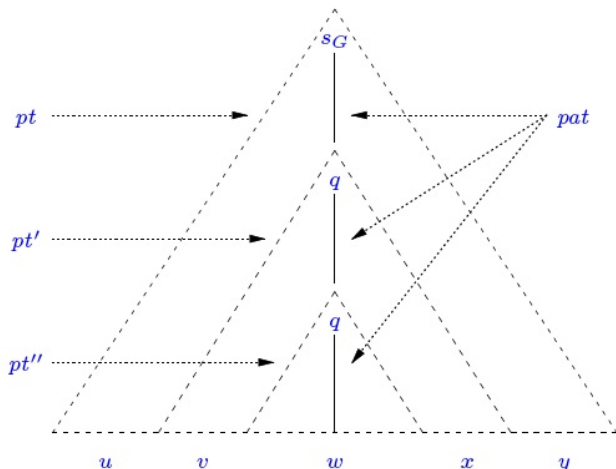
Lema do Bombeamento para LLC

Lema Seja L LC. Existe $k > 0$, tal que, para $\gamma \in L$, $|\gamma| \geq k$, existem $u, v, z, x, y \in \Sigma$ tal que $uvzxy = w$, $|vzx| \leq k$, $vx \neq \epsilon$ e para $i \in \mathbb{N}$, $uv^i z x^i y \in L$. **Prova** - G uma gramática LC para L , r o número de regras em G , $k > r + 1$.



Lema do Bombeamento para LLC

Lema Seja L LC. Existe $k > 0$, tal que, para $\gamma \in L$, $|\gamma| \geq k$, existem $u, v, z, x, y \in \Sigma$ tal que $uvzxy = w$, $|vzx| \leq k$, $vx \neq \epsilon$ e para $i \in \mathbb{N}$, $uv^i zx^i y \in L$. **Prova** - G uma gramática LC para L , r o número de regras em G , $k > r + 1$.



Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{w c w^R / w \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{w / \#a(w) = \#b(w)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{w_1 w_2 / w_1 \neq w_2, \text{ e } w_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{w c w^R / w \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{w / \#a(w) = \#b(w)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{w_1 w_2 / w_1 \neq w_2, \text{ e } w_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$